

Laporan Desain Rosetta Archive



Penulis:

Nisrina Zakiyah

18224001

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

1. Ringkasan

Rosetta Archive (RTA) adalah format arsip biner dengan payload di depan dan indeks di belakang. Saya memilih susunan ini karena proses ekstraksi dan akses acak mempunyai kebutuhan yang berbeda. Saat ekstraksi, reader diuntungkan jika payload tersusun berurutan. Untuk list, inspect, dan akses acak, reader membutuhkan indeks yang dapat ditemukan tanpa membaca seluruh payload. Header menunjuk indeks, footer mengulang lokasinya untuk pemulihan, dan setiap record indeks menunjuk langsung ke payload file.

Implementasi referensi ditulis dengan Python 3.10 tanpa dependency runtime. Enam perintah wajib tersedia: create, list, info, inspect, verify, dan extract. Implementasi juga menyediakan recover untuk menyalin entri yang masih valid ke arsip baru. Setiap file dapat dikompresi dengan zlib, sedangkan offset dalam indeks memungkinkan reader mengakses payload secara langsung.

2. Struktur dan representasi informasi

File RTA 1.0 memiliki empat region: header 112 byte, data file, indeks panjang variabel, dan footer 64 byte. Semua integer memakai little endian. Data direktori tidak mengambil ruang payload. Record file menyimpan path, tipe, mode, uid, gid, mtime nanosecond, ukuran asli, ukuran tersimpan, method, offset, dua CRC32, dan SHA-256.

Writer baru mengetahui seluruh ukuran dan offset setelah input selesai dibaca, sehingga indeks ditempatkan di akhir. Implementasi referensi menyimpan isi file di memori selama proses tersebut. Reader tidak perlu menyimpan seluruh payload di memori. Reader cukup membaca indeks, lalu melakukan seek ke payload file yang dipilih. Data saat verifikasi dan ekstraksi diproses per chunk 1 MiB.

Path disimpan sebagai UTF-8 dalam bentuk NFC dengan separator /. Urutan indeks ditentukan oleh byte UTF-8, bukan urutan directory traversal host. Pengurutan berdasarkan byte UTF-8 tidak bergantung pada locale host. Dengan input dan metadata yang sama, urutan record juga tetap sama. Direktori mempunyai record sendiri agar direktori kosong dan metadatanya tidak hilang.

3. Metadata dan rekonstruksi

Mode diperlukan untuk membedakan file biasa yang dapat dieksekusi dari file data biasa. mtime dipakai untuk mengembalikan waktu modifikasi. uid dan gid menyimpan kepemilikan sumber, walaupun extractor tidak menerapkannya secara default karena operasi itu sering membutuhkan hak administratif. Pengguna dapat memilih `--preserve-owner`.

Metadata tidak mencakup ACL, extended attribute, birth time, hard link, symlink, dan special file. Pada versi 1.0, saya membatasi tipe entri menjadi file biasa dan direktori agar aturan ekstraksinya tetap sederhana dan aman. Symlink menyulitkan model ekstraksi aman karena target dapat berada di luar output root. Writer menolak symlink karena targetnya dapat menunjuk ke luar output root. Dukungan symlink baru dapat ditambahkan setelah format mempunyai aturan ekstraksi yang aman untuk kasus tersebut.

Extractor menjalankan verifikasi penuh sebelum membuat output. Path absolut, komponen `..`, NUL, prefix drive, dan path noncanonical ditolak saat parsing. Saat menulis, extractor memeriksa kembali setiap komponen yang telah ada, menolak symlink, memakai create exclusive, dan tidak menimpa file. Metadata file diterapkan setelah isi selesai. Metadata direktori diterapkan dari direktori terdalam agar pembuatan child tidak mengubah mtime akhir parent.

4. Integritas dan ketahanan parser

RTA menggunakan checksum yang berbeda untuk struktur, payload tersimpan, dan isi setelah dekompresi. Satu checksum saja tidak dapat membedakan ketiga jenis kerusakan tersebut. Header, footer, dan setiap record indeks memiliki CRC32. Header menyimpan SHA-256 indeks lengkap, sedangkan footer menyimpan CRC32 indeks. Setiap file mempunyai CRC32 payload tersimpan, CRC32 isi asli, dan SHA-256 isi asli.

CRC32 dipilih untuk pemeriksaan cepat terhadap kerusakan tidak sengaja. SHA-256 digunakan untuk membandingkan isi asli file setelah dekompresi. CRC32 tetap digunakan untuk mendeteksi kerusakan secara cepat. Ini belum menjadi sistem autentikasi. Penyerang yang dapat menulis ulang

arsip masih dapat menghitung hash baru karena format belum memiliki tanda tangan digital.

Parser terlebih dahulu memeriksa header, footer, versi, flags, dan batas indeks. Setelah itu, parser memvalidasi ukuran record, path UTF-8, checksum, urutan entri, parent directory, metode penyimpanan, dan rentang payload. Rentang file harus berurutan tanpa gap atau overlap. Decoder memakai `original_size` sebagai batas keluaran. Jika hasil dekompresi melewati ukuran tersebut, reader menolak arsip. Stream yang truncated, menghasilkan data terlalu banyak, atau memiliki trailing bytes ditolak.

Kasus korupsi diuji secara langsung. Test suite mengubah byte payload, memotong akhir arsip, mengganti major version, dan membuat record `../x.txt` sambil menghitung ulang semua checksum struktur. Kasus terakhir tetap ditolak oleh aturan path. Pengujian juga memastikan ekstraksi belum membuat output ketika verifikasi gagal.

5. Versioning dan ekstensi

Versi disimpan sebagai pasangan major dan minor pada header, index header, dan footer. Reader 1.0 hanya menerima major 1 dan minor paling tinggi 0. Versi major baru dapat mengubah layout atau arti field, sehingga reader 1.0 tidak boleh membacanya sebagai format versi 1. Minor baru boleh mempertahankan layout, tetapi reader lama tetap berhenti jika belum memahami semantiknya.

Setiap record mempunyai `record_size`, `path_length`, dan extension area setelah path. Extension memakai TLV dengan type, flags, dan panjang payload. Extension optional yang belum dikenal boleh dilewati. Extension critical yang belum dikenal harus ditolak. CRC record mencakup TLV sehingga metadata tambahan tetap mendapat perlindungan yang sama.

Reserved field harus nol. Reader menolak reserved field yang tidak nol. Dengan begitu, writer tidak dapat menambahkan arti baru pada field tersebut tanpa memperbarui versi atau mendefinisikan extension. Identitas versi yang sama juga muncul di tiga struktur, dan reader meminta semuanya konsisten.

6. Determinisme

Writer tidak menulis waktu pembuatan atau UUID acak. `created_ns` selalu nol. Writer menormalisasi nama ke NFC dan menolak path yang bertabrakan. Setelah itu, entri diurutkan berdasarkan byte UTF-8. Writer memakai parameter zlib yang tetap dan hanya menyimpan hasil kompresi jika ukurannya lebih kecil. Penulisan menggunakan file sementara dan rename atomik, tetapi nama file sementara tidak masuk ke arsip.

Definisi input identik mencakup isi, nama, mode, uid, gid, mtime, opsi kompresi, implementasi, dan versi zlib. Dua run pada input tersebut menghasilkan byte yang sama. Metadata filesystem sengaja masuk definisi karena mengubah mtime atau permission memang mengubah arsip yang seharusnya direkonstruksi.

7. Fitur bonus: pemulihan, kompresi, dan random access

Footer mengulang offset serta ukuran indeks. recover dapat memulai dari akhir file walaupun header rusak. Tool menggunakan magic RENO untuk menemukan posisi calon record. Record tersebut baru diterima setelah seluruh struktur dan payload-nya lolos verifikasi. Calon record harus lolos CRC struktur, aturan path, batas data, CRC payload, dekompresi, ukuran hasil, CRC isi, dan SHA-256. Record yang gagal dilewati. Entri yang lolos ditulis ke arsip baru, sementara arsip sumber tidak berubah.

Pemulihan hanya menyelamatkan record yang masih dapat diverifikasi. Record yang rusak dilewati dan tidak dimasukkan ke arsip hasil. Footer yang rusak atau indeks yang hilang total tidak dapat dipulihkan oleh implementasi ini. Jika record direktori rusak tetapi child masih valid, tool membuat parent dengan mode 0755, uid/gid nol, dan mtime nol. Pilihan ini menjaga struktur tanpa berpura-pura mengetahui metadata yang sudah hilang.

Kompresi bekerja per file, bukan sebagai satu stream besar. File yang tidak cocok dikompresi tetap disimpan apa adanya. Kompresi per file membutuhkan metadata tambahan pada setiap record. Sebagai gantinya, reader dapat membaca atau memverifikasi satu file tanpa mendekompresi

payload lain. Kerusakan payload juga terisolasi, sehingga recovery masih dapat mengambil file lain.

8. Trade-off dan limitasi

Indeks akhir membuat listing dan random access cepat, tetapi writer perlu mengetahui semua offset sebelum finalisasi. Implementasi referensi menahan isi file di memori. Formatnya sendiri tidak mengharuskan hal tersebut. Writer untuk arsipbesar dapat melakukan dua pass atau memakai file sementara.

Pemeriksaan berlapis menambah 108 byte fixed metadata per entri, path, dan hash. Metadata 108 byte per record cukup besar untuk arsip yang berisi banyak file kecil. Metadata tersebut menyimpan checksum terpisah untuk struktur record, payload tersimpan, dan isi asli. Format ini memang tidak mengejar rasio kompresi terbaik.

RTA 1.0 belum mendukung append, deduplikasi, enkripsi, signature, sparse file, ACL, xattr, hard link, atau symlink. Batas parser referensi adalah satu juta entri, indeks 256 MiB, dan path 1 MiB. Batas tersebut mencegah alokasi yang tidak masuk akal ketika metadata corrupt. Pengembangan berikutnya dapat menambahkan tanda tangan digital melalui extension critical. Writer juga dapat diubah menjadi dua pass agar tidak perlu menyimpan seluruh isi file di memori.

9. Hasil pengujian

Sepuluh pengujian otomatis mencakup round trip file teks dan biner, direktori kosong, mode file, determinisme byte penuh, korupsi payload, truncation, versi tidak didukung, traversal path dengan checksum struktur valid, entri duplikat, offset di luar batas, recovery parsial, dan pemanggilan semua perintah wajib. Seluruh pengujian berjalan hanya dengan Python standard library.

RTA dirancang agar parser dapat memeriksa setiap ukuran dan offset sebelum membaca payload. Extractor juga menyelesaikan verifikasi penuh sebelum membuat file output. Dua aturan tersebut menjadi dasar keputusan desain pada implementasi versi 1.0.